Efficient Search Strategy for a Student Record System

Name: A. Lakshmi Gayathri

Reg No: 192411188

Seed: 1188

Git Hub: https://github.com/gayatri12765/DAA-A1-192411188-A.-Lakshmi-Gayathri.git

Task 1: Algorithm Design

#	Algorithm	Strategy	Style	Preprocessing
1	Linear search	Brute force	Iterative	None
2	Binary search	Decrease and conquer	Recursive	Sort, O(n log n)
3	Hash table (chaining)	Space-time trade-off (hashing)	Iterative	Build table, O(n)

Algorithm 1: Linear search. Scan records one by one until the key matches.

```
LinearSearch(A[0..n-1], key)
  for i <- 0 to n-1
    if A[i] = key return i
  return -1
```

Justification: baseline; needs no preprocessing; best for tiny or rarely searched data.

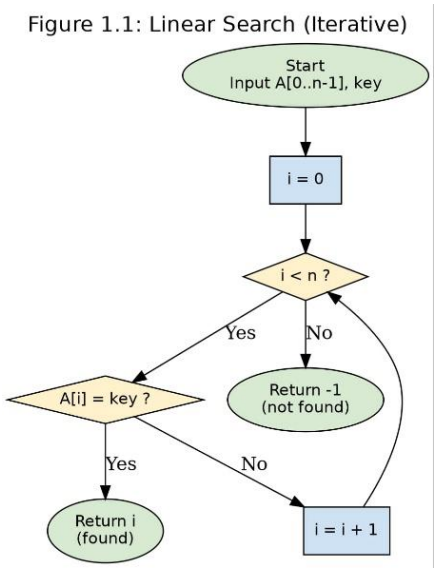


Fig: 1.1

Algorithm 2: Binary search (recursive). Compare the key with the middle element and discard half.

```
BinSearch(A, key, lo, hi)
  if lo > hi return -1
  mid <- floor((lo+hi)/2)
  if A[mid] = key return mid
  if key < A[mid] return BinSearch(A, key, lo, mid-1)
  else return BinSearch(A, key, mid+1, hi)
```

Preprocessing: sort the array once. Justification: registration numbers are orderable, so halving gives logarithmic search.

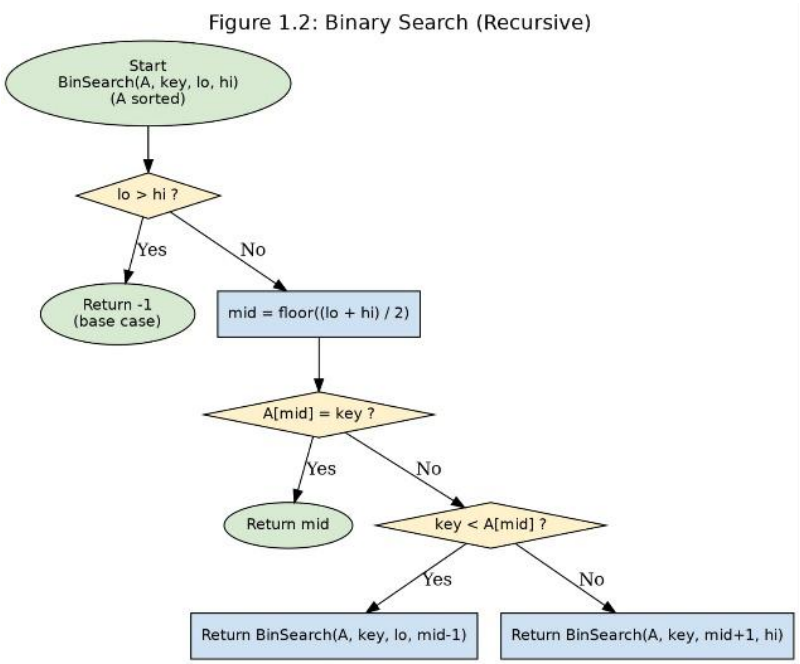


Fig: 1.2

Algorithm 3: Hash table with chaining. h(k) = k mod m, m = 2n+1; each bucket holds a list.

Build: for each k in A: T[h(k)].append(k)
Search(key): for each v in T[h(key)]: if v = key return found
return not found

Justification: trades memory for near-constant search; suits repeated lookups.

Figure 1.3: Hash Table Search (Chaining)

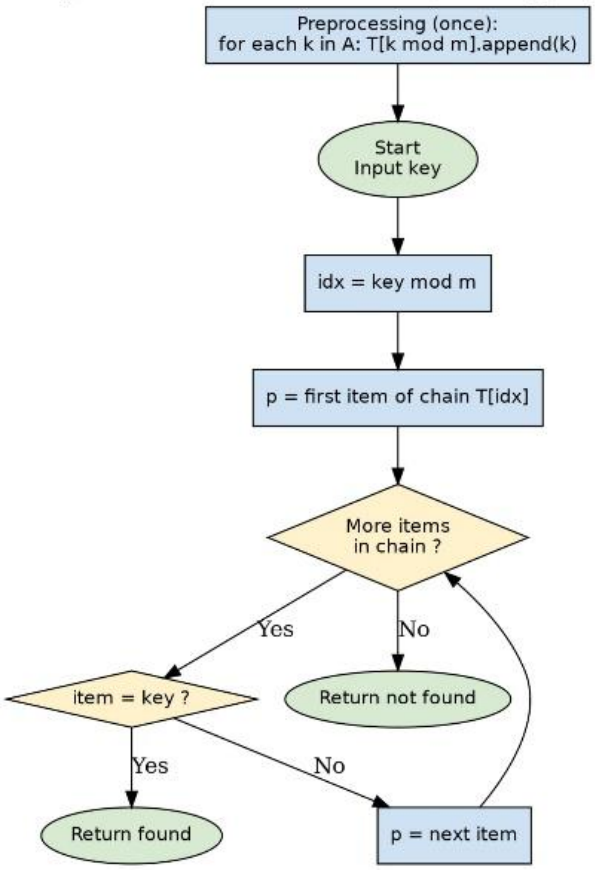


Fig: 1.3

Task 2: Basic Operation

Algorithm	Basic operation	Location	Growth with n
Linear	Comparison A[i] = key	Inside loop	Proportional to n
Binary	Comparison A[mid] = key	Each recursive call	+1 each time n doubles
Hash	Key comparison in bucket chain	Chain traversal	Depends on load factor, not n

Why: the comparison executes on every iteration or call and everything else is constant work around it, so it dominates running time.

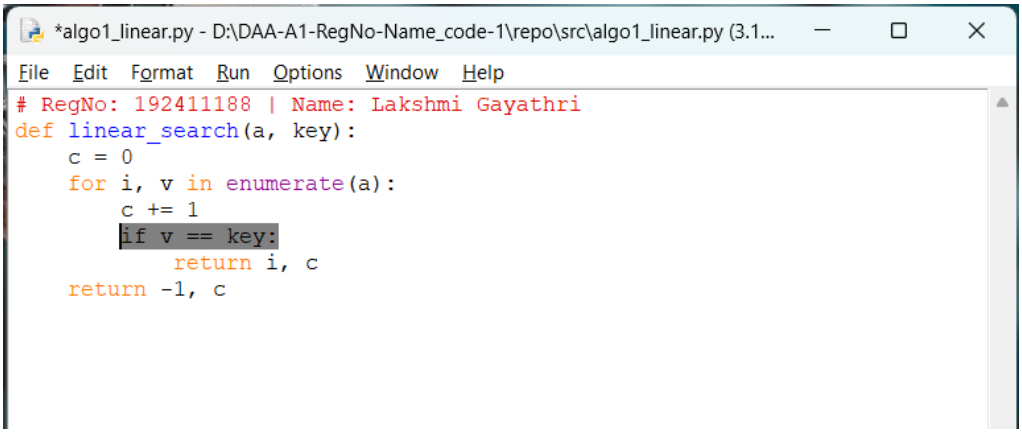


Fig: 2.1

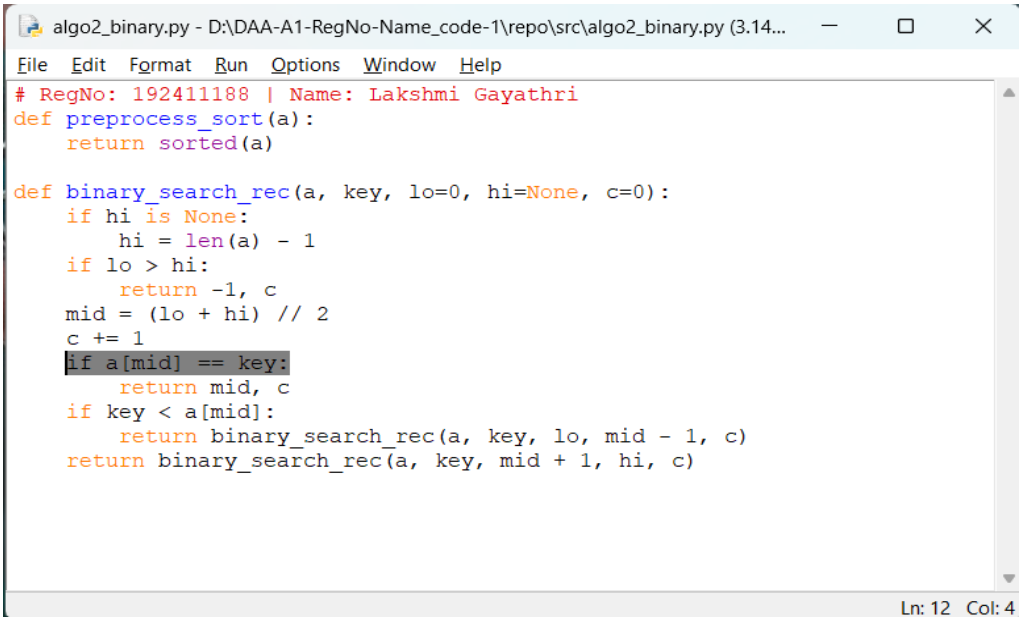


Fig: 2.2

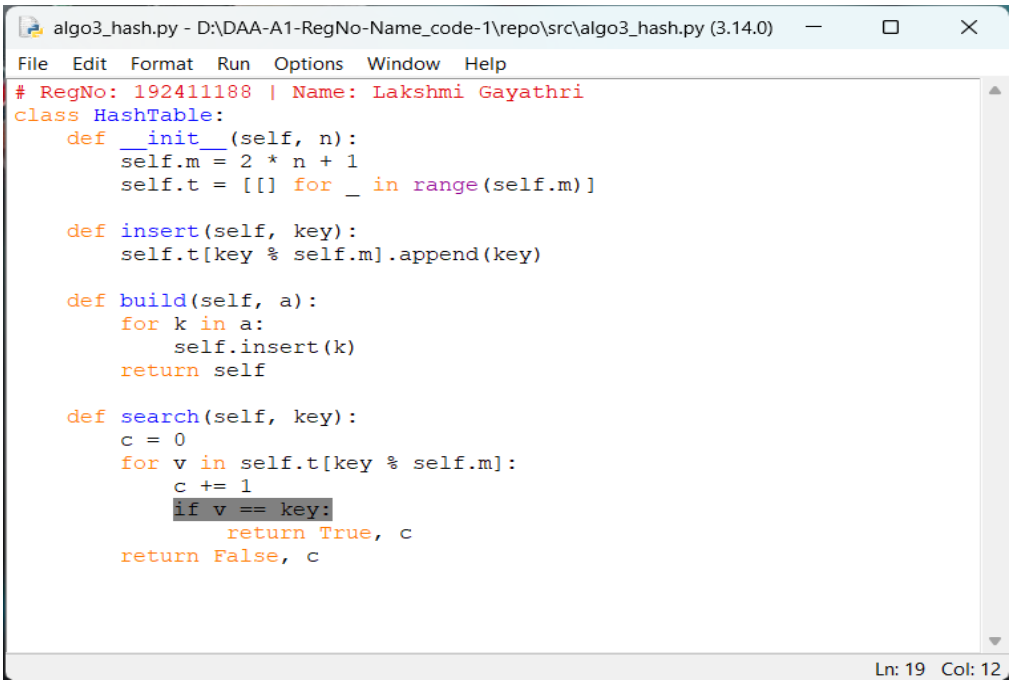


Fig:2.3

Task 3: Time-Complexity Derivation

Input size n = number of records.

Linear. Worst case C(n) = sum from i=0 to n-1 of 1 = n. Successful average, each position equally likely: C_avg = (1/n) * sum from i=1 to n of i = (n+1)/2. No preprocessing, so total = Theta(n) per search.

Binary. Each probe halves the interval: n, n/2, n/4, … The search stops when the interval size reaches 1 after k halvings, so n/2^k = 1 and k = log2 n. Worst case C(n) = floor(log2 n) + 1. Preprocessing (comparison sort) = Theta(n log n). For q searches: T(n,q) = n log n + q log n.

Hash. Load factor alpha = n/m ~ 0.5. Expected comparisons: successful ~ 1 + alpha/2, unsuccessful ~ alpha. Both constant. Build = n inserts of O(1) = Theta(n). For q searches: T(n,q) = n + q * Theta(1). Worst case (all keys in one bucket) = n.

Algorithm	Preprocessing	One search	q searches in total
Linear	0	n	q n
Binary	n log n	log n	n log n + q log n
Hash	n	1 (avg)	n + q

Task 4: Best, Average, Worst Case

Linear. Best: key at index 0, 1 comparison. Average (successful): (n+1)/2. Worst: key at the last position or absent, n comparisons. Reason: the scan stops only on a match, so cost depends on where the key sits.

Binary. Best: key at the first midpoint, 1 comparison. Average: about log2 n - 1. Worst: key absent or found on the last probe, floor(log2 n) + 1. Reason: the interval halves each step regardless of the key, so cases differ only in when the search stops.

Hash. Best: key alone in its bucket, 1 comparison. Average: 1 + alpha/2. Worst: all n keys collide, n comparisons. Reason: cost is the chain length; a good hash function keeps chains short.

Counter outputs for Reg No (found) and Reg No + 1 (not found) are in Task 9.

Task 5: Asymptotic Analysis

Algorithm	Best	Worst	Tight (typical)
Linear	Omega(1)	O(n)	Theta(n) average
Binary	Omega(1)	O(log n)	Theta(log n) average
Hash	Omega(1)	O(n)	Theta(1) expected

Growth order: $1 < \log n < n$.

n	Linear (n)	Binary (log2 n)	Hash (~1)
10^3	1,000	~10	~1
10^4	10,000	~13	~1
10^5	100,000	~17	~1
10^6	1,000,000	~20	~1

Interpretation: growing the database 1000 times makes linear search 1000 times slower, binary search only about 2 times slower (10 to 20 probes), and hash search essentially unchanged.

Task 6: Recurrence Relation (Binary Search)

- Original problem size: n (elements between lo and hi).
- Subproblem size: at most n/2 (only one half is searched).
- Recursive calls per level: 1 (never both halves).
- Work outside the call: one comparison with A[mid] plus index arithmetic, Theta(1).
- Base case: empty range (lo > hi), T(0) = 1 or T(1) = 1.

T(n) = T(n/2) + 1, T(1) = 1.

Derived directly from the code: the algorithm makes one call on half the range after doing constant work.

Task 7: Solving the Recurrence

Method 1: Iteration (substitution). Let n = 2^k.

T(n) = T(n/2) + 1 = T(n/4) + 2 = T(n/8) + 3 = … = T(n/2^k) + k.

When n/2^k = 1, k = log2 n, so T(n) = T(1) + log2 n = 1 + log2 n = Theta(log n).

Method 2: Master Theorem check. a = 1, b = 2, f(n) = 1 = Theta(n^0). n^(log_b a) = n^0 = 1. Since f(n) = Theta(n^(log_b a)), Case 2 applies: T(n) = Theta(log n).

Interpretation: the work grows by one step each time the data doubles, so the recursion depth is about 20 for one million records.

Task 8: Implementation and Testing

Language: Python. Files: algo1_linear.py, algo2_binary.py, algo3_hash.py, dataset_generator.py, test_correctness.py, benchmark.py.

Procedure: seed = last 4 digits of Reg No; a dataset of n unique registration numbers (1K, 10K, 100K, 1M) is generated once per size and shared by all three algorithms; 100 search keys per size (own Reg No, Reg No + 1, 98 random existing keys); the test script checks 45 keys per size including found, not found, first, last and middle elements.

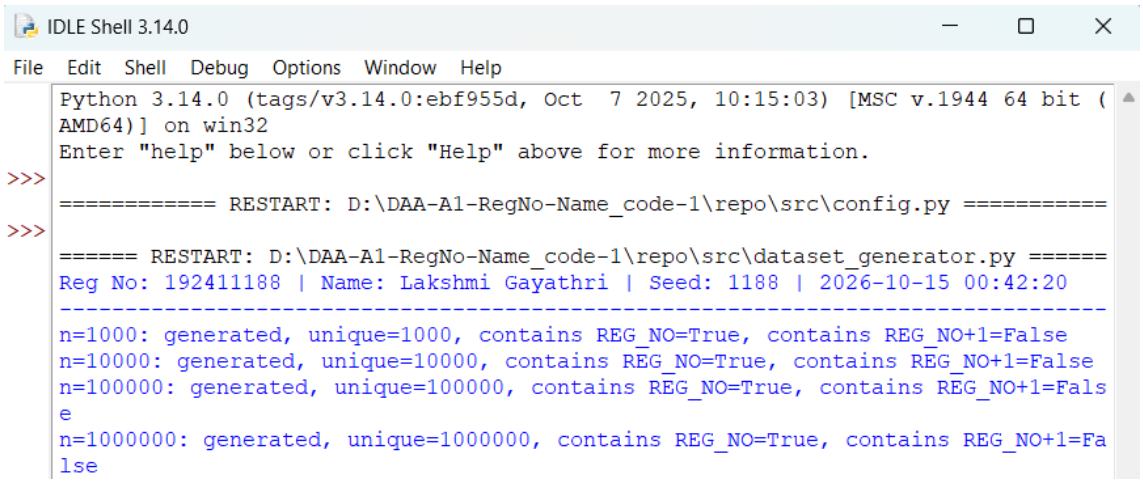


Fig: 8.1

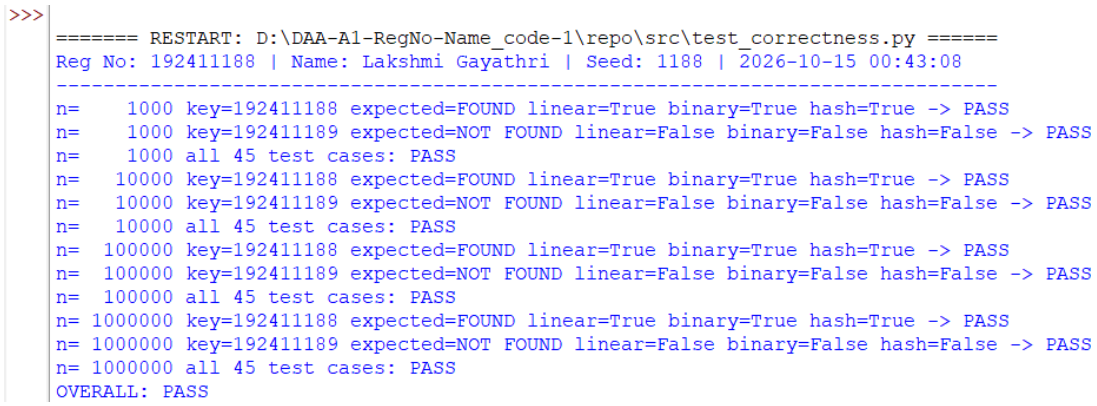


Fig: 8.2

Task 9: Experimental Performance

Command: python benchmark.py

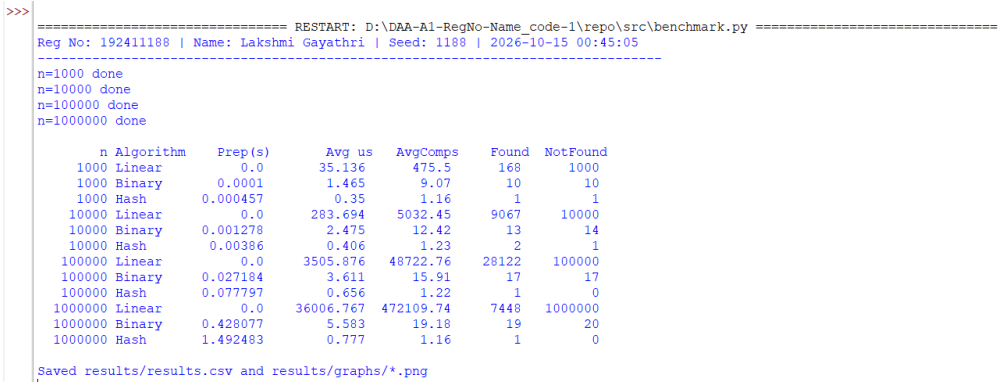


Fig: 9.1

```
>>>
============================== RESTART: D:\DAA-A1-RegNo-Name_code-1\repo\src\benchmark.py ==============================
Reg No: 192411188 | Name: Lakshmi Gayathri | Seed: 1188 | 2026-10-15 00:45:05
--------------------------------------------------------------------------------
n=1000 done
n=10000 done
n=100000 done
n=1000000 done

        n Algorithm    Prep(s)       Avg us   AvgComps    Found  NotFound
     1000 Linear          0.0       35.136      475.5      168      1000
     1000 Binary       0.0001        1.465       9.07       10        10
     1000 Hash       0.000457         0.35       1.16        1         1
    10000 Linear          0.0      283.694    5032.45     9067     10000
    10000 Binary     0.001278        2.475      12.42       13        14
    10000 Hash        0.00386        0.406       1.23        2         1
   100000 Linear          0.0     3505.876   48722.76    28122    100000
   100000 Binary     0.027184        3.611      15.91       17        17
   100000 Hash       0.077797        0.656       1.22        1         0
  1000000 Linear          0.0    36006.767  472109.74     7448   1000000
  1000000 Binary     0.428077        5.583      19.18       19        20
  1000000 Hash       1.492483        0.777       1.16        1         0

Saved results/results.csv and results/graphs/*.png
```

Fig: 9.2

```
>>>
============================== RESTART: D:\DAA-A1-RegNo-Name_code-1\repo\src\benchmark.py ==============================
Reg No: 192411188 | Name: Lakshmi Gayathri | Seed: 1188 | 2026-10-15 00:45:05
--------------------------------------------------------------------------------
n=1000 done
n=10000 done
n=100000 done
n=1000000 done

        n Algorithm    Prep(s)       Avg us   AvgComps    Found  NotFound
     1000 Linear          0.0       35.136      475.5      168      1000
     1000 Binary       0.0001        1.465       9.07       10        10
     1000 Hash       0.000457         0.35       1.16        1         1
    10000 Linear          0.0      283.694    5032.45     9067     10000
    10000 Binary     0.001278        2.475      12.42       13        14
    10000 Hash        0.00386        0.406       1.23        2         1
   100000 Linear          0.0     3505.876   48722.76    28122    100000
   100000 Binary     0.027184        3.611      15.91       17        17
   100000 Hash       0.077797        0.656       1.22        1         0
  1000000 Linear          0.0    36006.767  472109.74     7448   1000000
  1000000 Binary     0.428077        5.583      19.18       19        20
  1000000 Hash       1.492483        0.777       1.16        1         0

Saved results/results.csv and results/graphs/*.png
```

Fig: 9.3

```
>>>
============================== RESTART: D:\DAA-A1-RegNo-Name_code-1\repo\src\benchmark.py ==============================
Reg No: 192411188 | Name: Lakshmi Gayathri | Seed: 1188 | 2026-10-15 00:45:05
--------------------------------------------------------------------------------
n=1000 done
n=10000 done
n=100000 done
n=1000000 done

        n Algorithm    Prep(s)       Avg us   AvgComps    Found  NotFound
     1000 Linear          0.0       35.136      475.5      168      1000
     1000 Binary       0.0001        1.465       9.07       10        10
     1000 Hash       0.000457         0.35       1.16        1         1
    10000 Linear          0.0      283.694    5032.45     9067     10000
    10000 Binary     0.001278        2.475      12.42       13        14
    10000 Hash        0.00386        0.406       1.23        2         1
   100000 Linear          0.0     3505.876   48722.76    28122    100000
   100000 Binary     0.027184        3.611      15.91       17        17
   100000 Hash       0.077797        0.656       1.22        1         0
  1000000 Linear          0.0    36006.767  472109.74     7448   1000000
  1000000 Binary     0.428077        5.583      19.18       19        20
  1000000 Hash       1.492483        0.777       1.16        1         0

Saved results/results.csv and results/graphs/*.png
```

Fig: 9.4

Table 9.1: Average search time (microseconds) and preprocessing (s)

n	Linear time	Binary time	Hash time	Sort (s)	Hash build (s)
1,000	35.136	1.465	0.35	0.0001	0.000457
10,000	283.694	2.475	0.406	0.001278	0.00386
100,000	3505.876	3.611	0.656	0.027184	0.077797
1,000,000	36006.767	5.583	0.777	0.428077	1.492483

Table 9.2: Average comparisons (found keys)

n	Linear	Binary	Hash
1,000	475.5	9.07	1.16
10,000	5032.45	12.42	1.23
100,000	48722.76	15.91	1.22
1,000,000	472109.74	19.18	1.16

Table 9.3: Own keys. Comparisons for Reg No (found) and Reg No + 1 (not found) per algorithm and size:

n	**Linear**	**Binary**	**Hash**
1,000	168 / 1000	10 / 10	1 / 1
10,000	9067 / 10000	13 / 14	2 / 1
100,000	28122 / 100000	17 / 17	1 / 0
1,000,000	7448 / 1000000	19 / 20	1 / 0

Note: hash shows 0 comparisons when the key's bucket is empty, so nothing was compared.

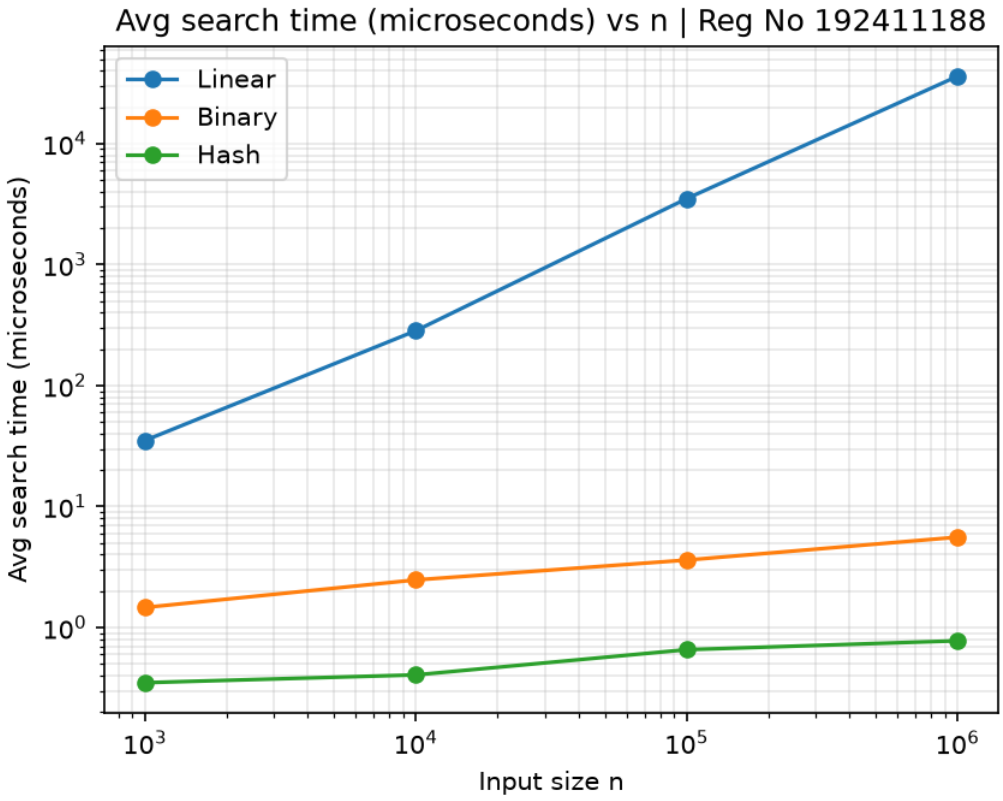

Avg search time (microseconds) vs n | Reg No 192411188
Linear
Binary
Hash
Avg search time (microseconds)
Input size n

Fig: 9.5

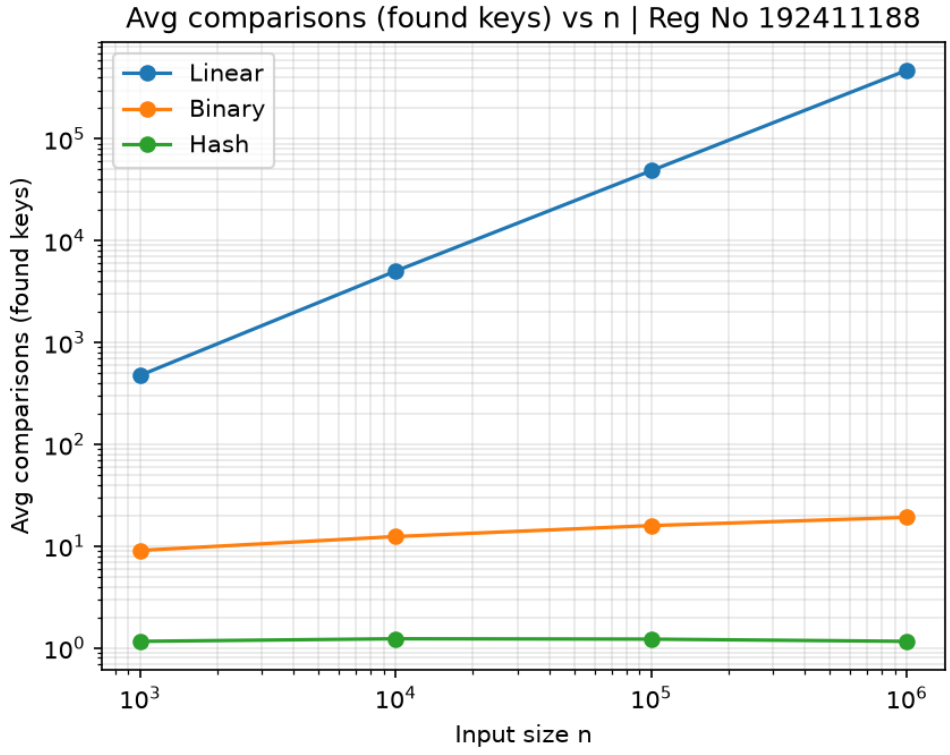

Avg comparisons (found keys) vs n | Reg No 192411188
Linear
Binary
Hash
Avg comparisons (found keys)
Input size n

Fig: 9.6

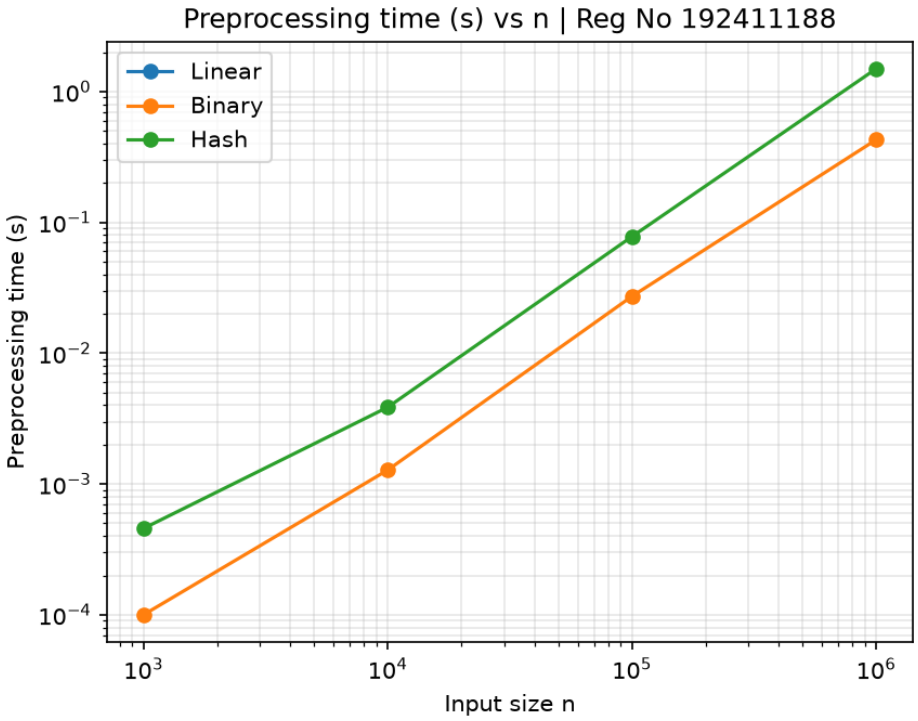

Preprocessing time (s) vs n | Reg No 192411188
Linear
Binary
Hash
Preprocessing time (s)
Input size n
10^{0}
10^{-1}
10^{-2}
10^{-3}
10^{-4}
10^{3}
10^{4}
10^{5}
10^{6}

Fig: 9.7

Task 10: Theory vs Experiment

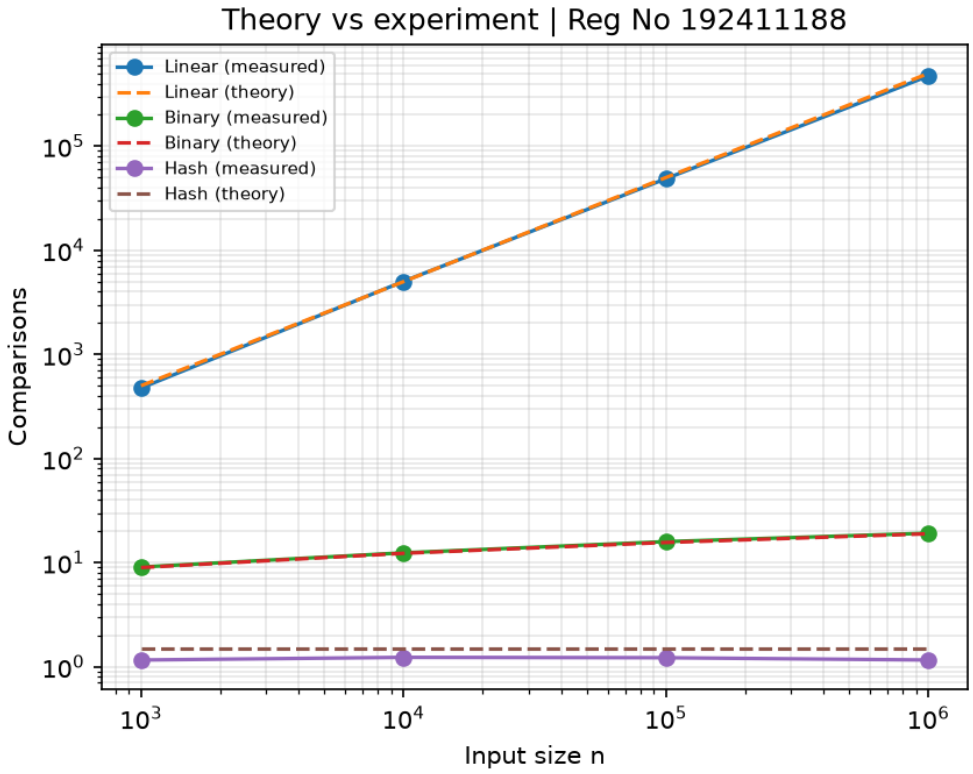


Fig: 10.1

Discuss in your own words, using your numbers:

- Linear: measured average comparisons (472,110 at 1M) are close to the predicted (n+1)/2 ≈ 500,000. The gap comes from using only 98 random keys. Time rose about 8, 12 and 10 times for each 10× increase in n, which fits Θ(n).
- Binary: measured 9.07, 12.42, 15.91, 19.18 match $\log_2 n - 1$ (8.97, 12.29, 15.61, 18.93). A 1000× larger database costs only about 3.8× more time (1.465 → 5.583 µs).
- Hash: comparisons stay between 1.16 and 1.23 at every size, matching the predicted $1 + \alpha/2 = 1.25$ for $\alpha \approx 0.5$.
- Differences: time per comparison is not constant. Python interpreter overhead, function-call cost (binary is recursive) and CPU cache misses at 1M records explain why hash time grew from 0.35 to 0.777 µs while its comparisons stayed flat.
- Best scaling: hash search, then binary search. Linear scales worst.

Task 11: Final Recommendation

- Recommended: binary search on sorted data for the original scenario.
- Theory: one sort of Θ(n log n), then Θ(log n) per search.
- Experiment: at 1M records, about 19 comparisons and 5.583 µs per search, against 472,110 comparisons and 36,006.767 µs for linear.
- Preprocessing: sorting takes 0.428 s. Break-even against linear is 0.428 s ÷ (36,006.767 − 5.583 µs) ≈ 12 searches, so it pays for itself almost immediately.
- Trade-offs: inserting into a sorted array costs O(n) or needs a re-sort. Hash gives faster search but uses about 2n buckets, takes 1.49 s to build, and has no ordering or range queries. Linear is only sensible for very small n.
- Practicality: with 1M records, 20 comparisons is still tiny, and doubling the database adds only about 1 comparison.

Task 12: Day-4 Design Challenge

A. Impact Analysis

- Change: a known, sustained load of 50,000 searches per hour (about 13.9 per second) on n = 1,000,000; updates only once a day.
- Affected assumption: the original analysis treated preprocessing and a single search separately; now one preprocessing step is shared by a huge number of searches.
- Why search frequency matters: total cost is preprocessing + q * per-search cost, so per-search cost dominates when q is large.
- Why rare updates matter: the structure can be rebuilt once a day, so costly preprocessing is amortised and insert cost barely matters.

B. Evaluation of the Original Design

Total work for q = 50,000 searches at n = 10^6:

- Linear: q * n/2 = about 2.5 x 10^10 comparisons; not acceptable.
- Binary: n log n + q log n = about 2 x 10^7 + 10^6 operations; acceptable.
- Hash: n + q = about 1.05 x 10^6 operations; lowest.

Preprocessing for sorting or hashing is paid once per day, so it is acceptable. Hash gives the better overall performance for exact-match lookup.

C. Adaptation or Redesign

Decision: **Modify**. Keep sorting and search for ordered or range needs, but serve exact lookups from a hash table rebuilt once a day after new records are added. Complexity changes from Θ(log n) per search to Θ(1) expected per search, with Θ(n) preprocessing per day.

Benchmark of revised design (python task12.py, 50,000 searches on 1M records):

```
>>>
================================ RESTART: D:\DAA-A1-RegNo-Name_code-1\repo\src\task12.py ================================
Reg No: 192411188 | Name: Lakshmi Gayathri | Seed: 1188 | 2026-10-15 00:48:23
--------------------------------------------------------------------------------
1000000 records, 50000 searches | required rate: 13.89 searches/s
Design                                    Prep(s)  50k total(s)  Per search(us)  Avg comps  Prep+50k(s)
Linear (extrapolated from 200 searches)    0.0000     2719.9694       54399.388  747469.70    2719.9694
Binary (sort once)                         0.3416        0.2079           4.158      19.46       0.5495
Hash (build once)                          1.0515        0.0422           0.844       0.85       1.0937
>>>
```

Fig: 12.1

Design	Preprocess (s)	50,000 searches (s)	Per search (us)
Linear (extrapolated from 200 searches)	0.0000	2719.9694	54399.388
Binary	0.3416	0.2079	4.158
Hash	1.0515	0.0422	0.844

D. Evidence-Based Defence

Combine in your own words:

- Complexity: Θ(n + q) for hash, Θ(n log n + q log n) for binary, Θ(qn) for linear.
- The required rate of 13.9 searches per second is far below the roughly 1.3 million per second that hash achieves in your measurements.
- The 1.49 s daily rebuild is negligible against a 24-hour window.
- Break-even against linear is about 42 searches (1.492 s ÷ 0.036 s), so the 50,000 hourly searches repay it immediately.
- Trade-offs: extra memory, no range queries, and a worst case of Θ(n) with a poor hash function, reduced by using m = 2n + 1 buckets.
- Final decision: modify to a daily-rebuilt hash table.